

Name: Muhammad Abu Bakar

NUID: 24K-0826

Section: BCS-243D

Syllabus: Database Systems Lab

Syllabus Code: CL2005

Instructor: Sir Osama Tahir

Q1)

SQL Query:

```
SELECT first_name, last_name, hire_date
FROM HR.employees
WHERE hire_date > (
    SELECT hire_date
    FROM HR.employees
    WHERE first_name = 'Nancy' AND last_name = 'Greenberg'
);
```

Snapshot of Query:

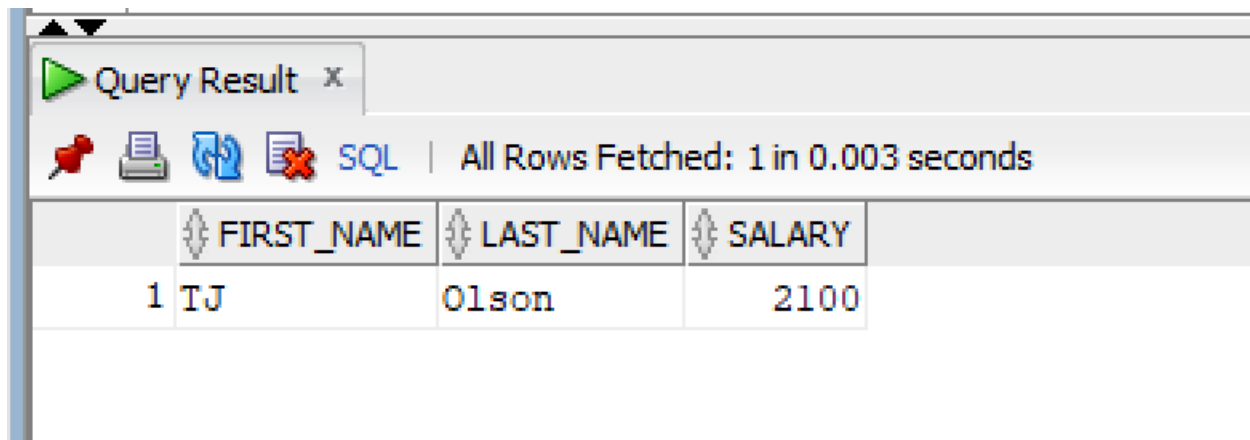
Query Result x				
SQL Fetched 50 rows in 0.006 seconds				
	FIRST_NAME	LAST_NAME	HIRE_DATE	
1	Steven	King	17-JUN-03	
2	Neena	Kochhar	21-SEP-05	
3	Alexander	Hunold	03-JAN-06	
4	Bruce	Ernst	21-MAY-07	
5	David	Austin	25-JUN-05	
6	Valli	Pataballa	05-FEB-06	
7	Diana	Lorentz	07-FEB-07	
8	John	Chen	28-SEP-05	
9	Ismael	Sciarra	30-SEP-05	
10	Jose Manuel	Urman	07-MAR-06	
11	Luis	Popp	07-DEC-07	
12	Den	Raphaely	07-DEC-02	
13	Alexander	Khoo	18-MAY-03	
14	Shelli	Baida	24-DEC-05	
15	Sigal	Tobias	24-JUL-05	
16	Guy	Himuro	15-NOV-06	
17	Karen	Colmenares	10-AUG-07	
18	Matthew	Weiss	18-JUL-04	
19	Adam	Fripp	10-APR-05	
20	Payam	Kaufling	01-MAY-03	
21	Shanta	Vollman	10-OCT-05	
22	Kevin	Mourgos	16-NOV-07	
23	Julia	Nayer	16-JUL-05	

Q2)

SQL Query:

```
SELECT first_name, last_name, salary
FROM HR.employees
WHERE salary = (
    SELECT MIN(salary)
    FROM HR.employees
);
```

Snapshot of Query:



The screenshot shows a 'Query Result' window with a toolbar containing icons for a pin, printer, refresh, and close. The status bar indicates 'All Rows Fetched: 1 in 0.003 seconds'. The query result is displayed in a table with three columns: FIRST_NAME, LAST_NAME, and SALARY. The first row contains the values 'TJ', 'Olson', and '2100'.

	FIRST_NAME	LAST_NAME	SALARY
1	TJ	Olson	2100

Q3)

SQL Query:

```
SELECT first_name, last_name, salary
FROM HR.employees
WHERE salary > (
    SELECT salary
    FROM HR.employees
    WHERE first_name = 'Karen' AND last_name = 'Colmenares'
);
```

Snapshot of Query:

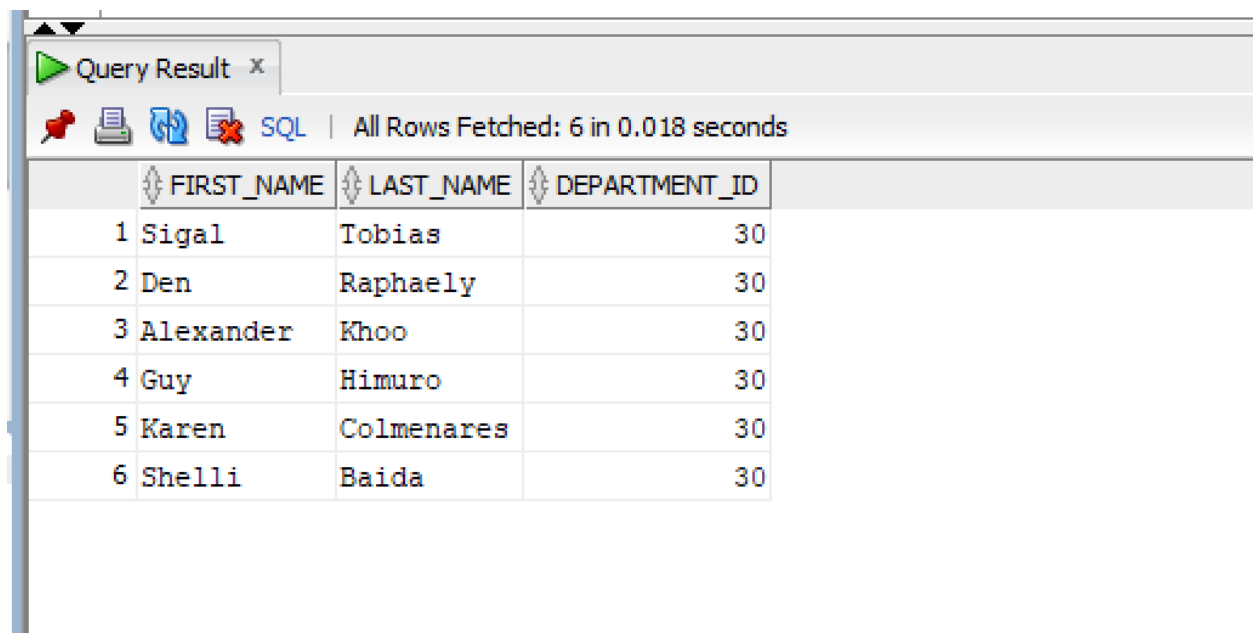
Query Result x			
SQL Fetched 50 rows in 0.01 seconds			
	FIRST_NAME	LAST_NAME	SALARY
1	Steven	King	24000
2	Neena	Kochhar	17000
3	Lex	De Haan	17000
4	Alexander	Hunold	9000
5	Bruce	Ernst	6000
6	David	Austin	4800
7	Valli	Pataballa	4800
8	Diana	Lorentz	4200
9	Nancy	Greenberg	12008
10	Daniel	Faviet	9000
11	John	Chen	8200
12	Ismael	Sciarra	7700
13	Jose Manuel	Urman	7800
14	Luis	Popp	6900
15	Den	Raphaely	11000
16	Alexander	Khoo	3100
17	Shelli	Baida	2900
18	Sigal	Tobias	2800
19	Guy	Himuro	2600
20	Matthew	Weiss	8000
21	Adam	Fripp	8200
22	Payam	Kaufling	7900
23	Shanta	Vollman	6500

Q4)

SQL Query:

```
SELECT first_name, last_name, department_id
FROM HR.employees
WHERE department_id IN (
    SELECT department_id
    FROM HR.employees
    WHERE job_id = 'PU_CLERK'
);
```

Snapshot of Query:



The screenshot shows a 'Query Result' window with a toolbar containing icons for a pin, printer, refresh, and error, along with an 'SQL' label. The status bar indicates 'All Rows Fetched: 6 in 0.018 seconds'. The table below displays the query results with columns FIRST_NAME, LAST_NAME, and DEPARTMENT_ID.

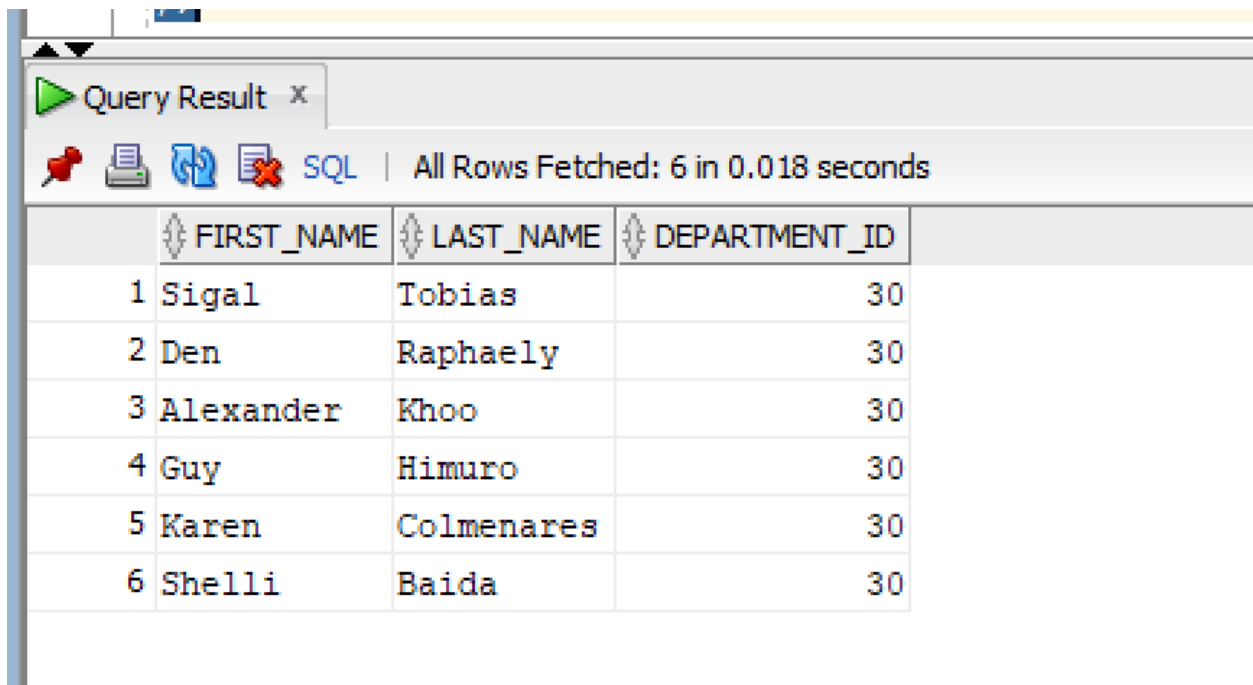
	FIRST_NAME	LAST_NAME	DEPARTMENT_ID
1	Sigal	Tobias	30
2	Den	Raphaely	30
3	Alexander	Khoo	30
4	Guy	Himuro	30
5	Karen	Colmenares	30
6	Shelli	Baida	30

Q5)

SQL Query:

```
SELECT first_name, last_name, job_id
FROM HR.employees
WHERE job_id IN (
    SELECT job_id
    FROM HR.employees
    WHERE department_id = 50
);
```

Snapshot of Query:



The screenshot shows a SQL query result window titled "Query Result x". The window displays the results of the query in a table format. The table has four columns: "FIRST_NAME", "LAST_NAME", and "DEPARTMENT_ID". The results are as follows:

	FIRST_NAME	LAST_NAME	DEPARTMENT_ID
1	Sigal	Tobias	30
2	Den	Raphaely	30
3	Alexander	Khoo	30
4	Guy	Himuro	30
5	Karen	Colmenares	30
6	Shelli	Baida	30

Q6)

SQL Query:

```
SELECT first_name, last_name, salary
FROM HR.employees
WHERE salary > ANY (
    SELECT salary
    FROM HR.employees
    WHERE department_id = 60
);
```


Snapshot of Query:

Query Result x				
    SQL Fetched 50 rows in 0.009 seconds				
	FIRST_NAME	LAST_NAME	SALARY	
1	Steven	King	24000	
2	Neena	Kochhar	17000	
3	Lex	De Haan	17000	
4	John	Russell	14000	
5	Karen	Partners	13500	
6	Michael	Hartstein	13000	
7	Nancy	Greenberg	12008	
8	Shelley	Higgins	12008	
9	Alberto	Errazuriz	12000	
10	Lisa	Ozer	11500	
11	Den	Raphaely	11000	
12	Gerald	Cambrault	11000	
13	Ellen	Abel	11000	
14	Eleni	Zlotkey	10500	
15	Clara	Vishney	10500	
16	Janette	King	10000	
17	Peter	Tucker	10000	
18	Hermann	Baer	10000	
19	Harrison	Bloom	10000	
20	Tayler	Fox	9600	
21	Danielle	Greene	9500	
22	David	Bernstein	9500	
23	Patrick	Sully	9500	

Q7)

SQL Query:

```
SELECT first_name, last_name, salary
FROM HR.employees
WHERE salary > ALL (
    SELECT salary
    FROM HR.employees
    WHERE department_id = 90
);
```

Snapshot of Query:

Query Result x			
			 SQL All Rows Fetched: 0 in 0.003 seconds
FIRST_NAME	LAST_NAME	SALARY	

Q8)

SQL Query:

```
SELECT first_name, last_name, department_id
FROM HR.employees
WHERE department_id IN (
    SELECT department_id
    FROM HR.employees
    GROUP BY department_id
    HAVING COUNT(employee_id) > 1
);
```

Snapshot of Query:

Query Result x			
SQL Fetched 50 rows in 0.004 seconds			
	FIRST_NAME	LAST_NAME	DEPARTMENT_ID
1	Alexander	Hunold	60
2	Valli	Pataballa	60
3	Jose Manuel	Urman	100
4	Guy	Himuro	30
5	Adam	Fripp	50
6	TJ	Olson	50
7	Jason	Mallin	50
8	Hazel	Philtanker	50
9	Peter	Vargas	50
10	Karen	Partners	80
11	David	Bernstein	80
12	Peter	Hall	80
13	Jonathon	Taylor	80
14	Girard	Geoni	50
15	Alexis	Bull	50
16	Timothy	Gates	50
17	Sarah	Bell	50
18	Douglas	Grant	50
19	Pat	Fay	20
20	William	Gietz	110
21	Lex	De Haan	90
22	Bruce	Ernst	60
23	Daniel	Faviet	100

Q9)

SQL Query:

```
SELECT first_name, last_name, salary, department_id
FROM HR.employees e
WHERE salary > (
    SELECT AVG(salary)
    FROM HR.employees
    WHERE department_id = e.department_id
);
```

Snapshot of Query:

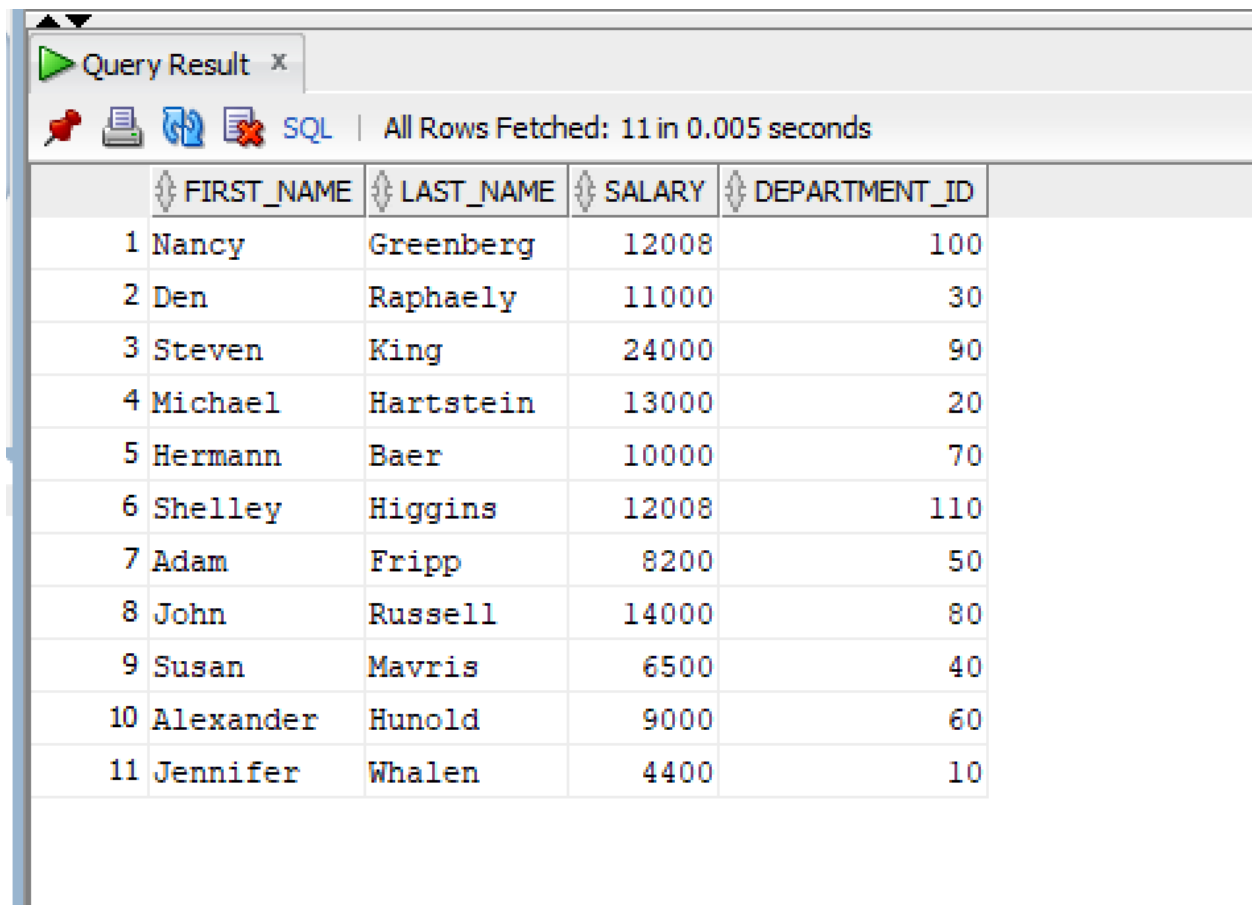
Query Result x				
All Rows Fetched: 38 in 0.004 seconds				
	FIRST_NAME	LAST_NAME	SALARY	DEPARTMENT_ID
1	Steven	King	24000	90
2	Alexander	Hunold	9000	60
3	Bruce	Ernst	6000	60
4	Nancy	Greenberg	12008	100
5	Daniel	Faviet	9000	100
6	Den	Raphaely	11000	30
7	Matthew	Weiss	8000	50
8	Adam	Fripp	8200	50
9	Payam	Kaufling	7900	50
10	Shanta	Vollman	6500	50
11	Kevin	Mourgos	5800	50
12	Renske	Ladwig	3600	50
13	Trenna	Rajs	3500	50
14	John	Russell	14000	80
15	Karen	Partners	13500	80
16	Alberto	Errazuriz	12000	80
17	Gerald	Cambrault	11000	80
18	Eleni	Zlotkey	10500	80
19	Peter	Tucker	10000	80
20	David	Bernstein	9500	80
21	Peter	Hall	9000	80
22	Janette	King	10000	80
23	Patrick	Sully	9500	80

Q10)

SQL Query:

```
SELECT first_name, last_name, salary, department_id
FROM HR.employees
WHERE (department_id, salary) IN (
    SELECT department_id, MAX(salary)
    FROM HR.employees
    GROUP BY department_id
);
```

Snapshot of Query:



The screenshot shows a 'Query Result' window with a toolbar containing icons for a pin, printer, refresh, and close. The status bar indicates 'All Rows Fetched: 11 in 0.005 seconds'. The table below displays the results of the SQL query, with columns for row number, first name, last name, salary, and department ID.

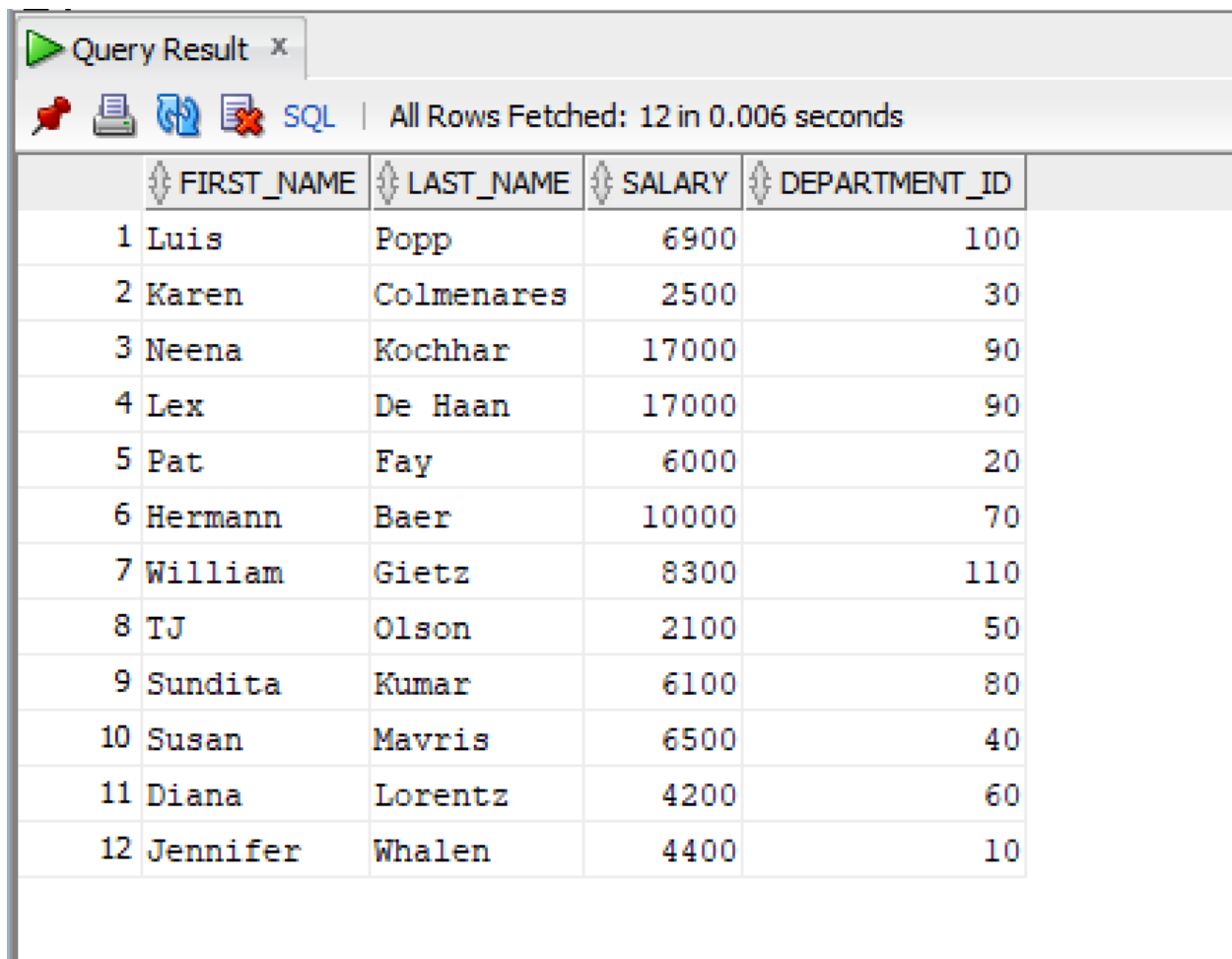
	FIRST_NAME	LAST_NAME	SALARY	DEPARTMENT_ID
1	Nancy	Greenberg	12008	100
2	Den	Raphaely	11000	30
3	Steven	King	24000	90
4	Michael	Hartstein	13000	20
5	Hermann	Baer	10000	70
6	Shelley	Higgins	12008	110
7	Adam	Fripp	8200	50
8	John	Russell	14000	80
9	Susan	Mavris	6500	40
10	Alexander	Hunold	9000	60
11	Jennifer	Whalen	4400	10

Q11)

SQL Query:

```
SELECT first_name, last_name, salary, department_id
FROM HR.employees
WHERE (department_id, salary) IN (
    SELECT department_id, MIN(salary)
    FROM HR.employees
    GROUP BY department_id
);
```

Snapshot of Query:



The screenshot shows a SQL query result window titled "Query Result x". It displays 12 rows of data from the HR.employees table, ordered by department_id and then by salary. The columns are FIRST_NAME, LAST_NAME, SALARY, and DEPARTMENT_ID. The status bar indicates "All Rows Fetched: 12 in 0.006 seconds".

	FIRST_NAME	LAST_NAME	SALARY	DEPARTMENT_ID
1	Luis	Popp	6900	100
2	Karen	Colmenares	2500	30
3	Neena	Kochhar	17000	90
4	Lex	De Haan	17000	90
5	Pat	Fay	6000	20
6	Hermann	Baer	10000	70
7	William	Gietz	8300	110
8	TJ	Olson	2100	50
9	Sundita	Kumar	6100	80
10	Susan	Mavris	6500	40
11	Diana	Lorentz	4200	60
12	Jennifer	Whalen	4400	10

Q12)

SQL Query:

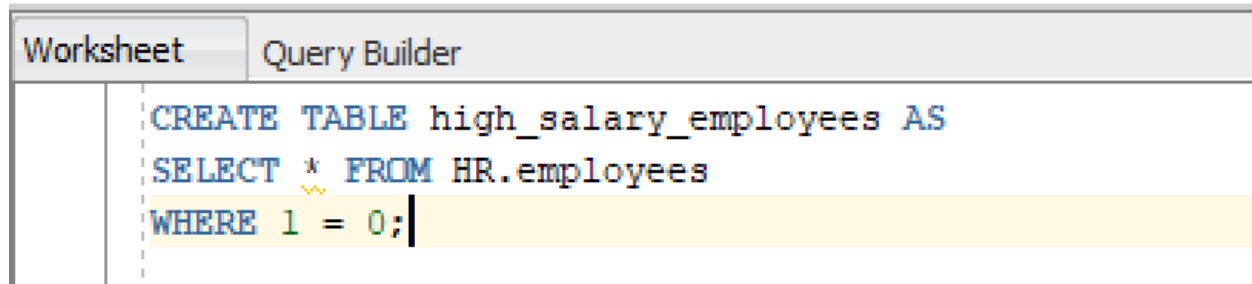
CREATING THE TABLE

```
CREATE TABLE high_salary_employees AS  
SELECT * FROM HR.employees  
WHERE 1 = 0;
```

INSERTING DATA USING SUB QUERY

```
INSERT INTO high_salary_employees  
SELECT *  
FROM HR.employees  
WHERE salary > 10000;
```

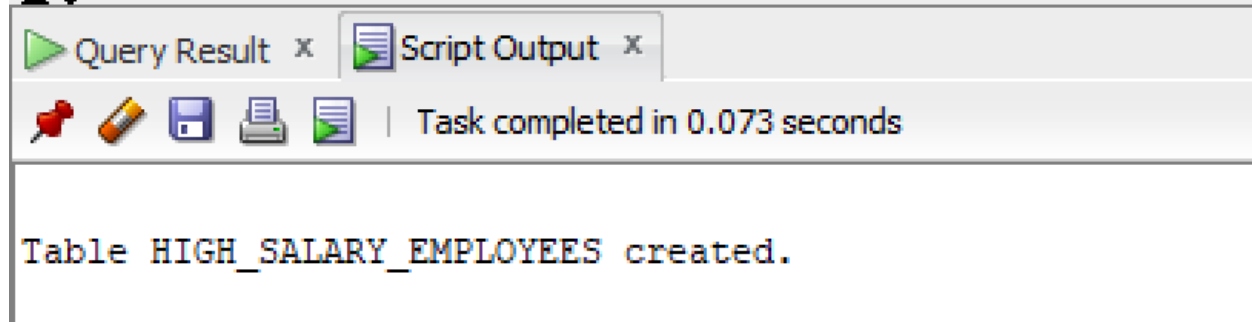
Snapshot of Query:



The screenshot shows a SQL Query Builder window with two tabs: 'Worksheet' and 'Query Builder'. The 'Query Builder' tab is active, displaying the following SQL query:

```
CREATE TABLE high_salary_employees AS  
SELECT * FROM HR.employees  
WHERE 1 = 0;
```

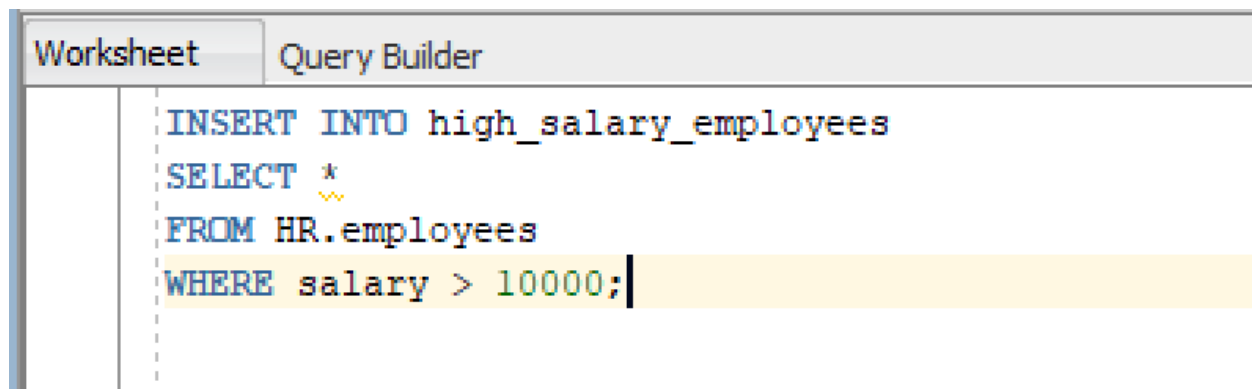
The query is highlighted in yellow. The 'Worksheet' tab is visible but empty.



The screenshot shows the execution result of the query. The 'Query Result' tab is active, displaying the message:

```
Table HIGH_SALARY_EMPLOYEES created.
```

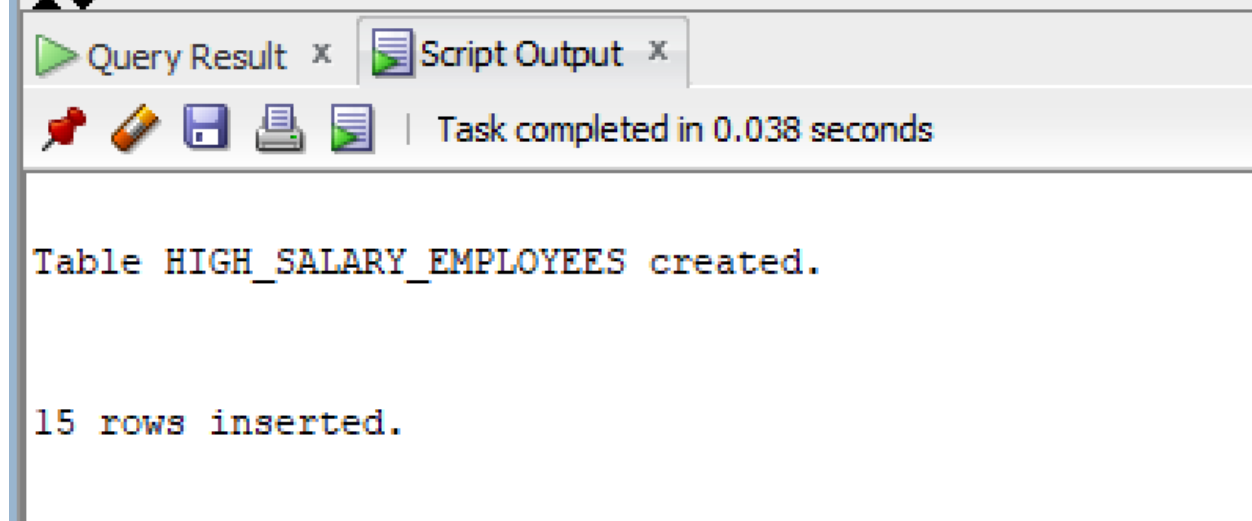
The 'Script Output' tab is also visible, showing the same message. The status bar indicates 'Task completed in 0.073 seconds'.



The screenshot shows a SQL Query Builder window with two tabs: 'Worksheet' and 'Query Builder'. The 'Query Builder' tab is active, displaying the following SQL query:

```
INSERT INTO high_salary_employees  
SELECT *  
FROM HR.employees  
WHERE salary > 10000;
```

The query is highlighted in yellow. The 'Worksheet' tab is visible but empty.



The screenshot shows the execution result of the query. The 'Query Result' tab is active, displaying the message:

```
Table HIGH_SALARY_EMPLOYEES created.  
  
15 rows inserted.
```

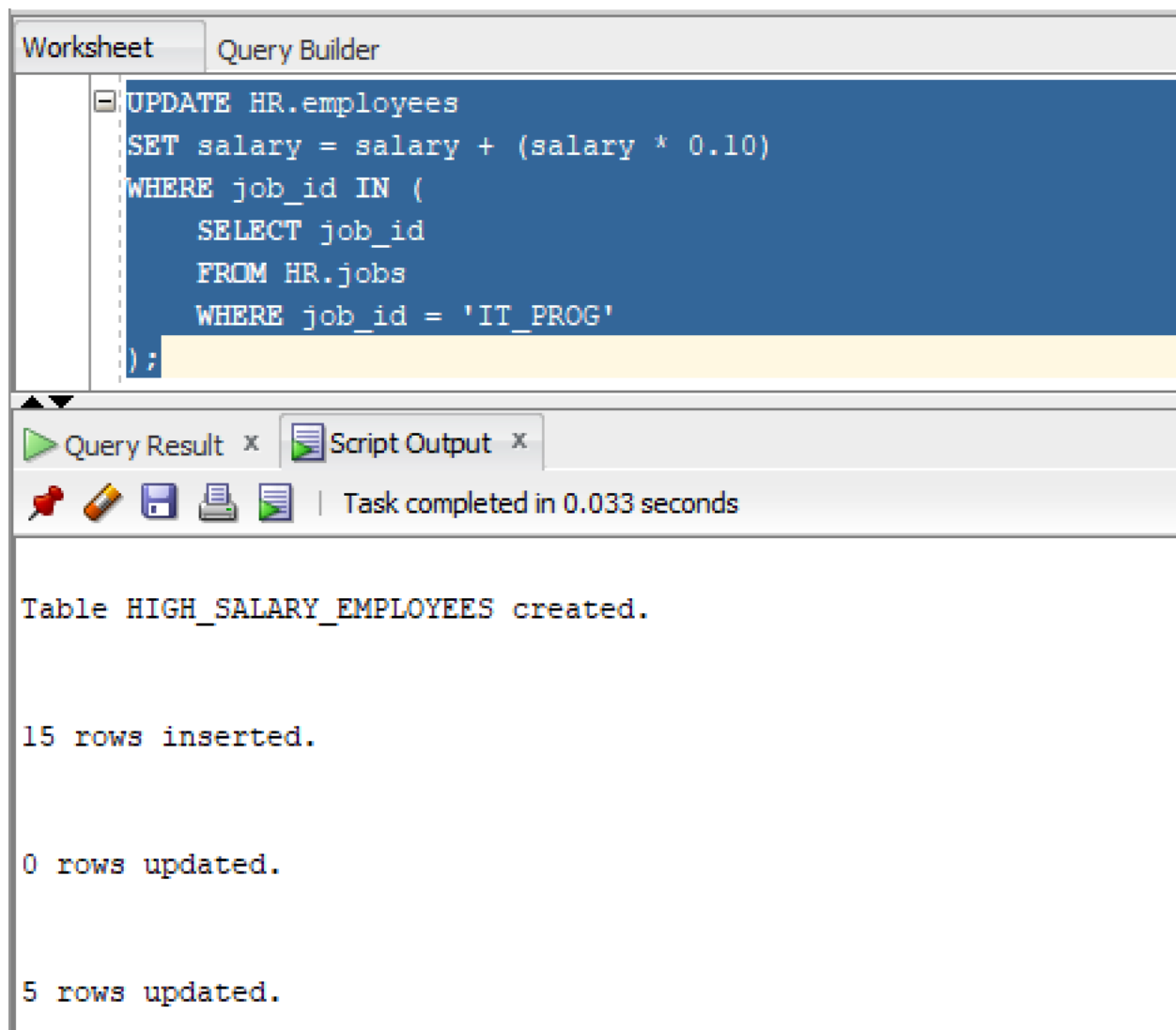
The 'Script Output' tab is also visible, showing the same message. The status bar indicates 'Task completed in 0.038 seconds'.

Q13)

SQL Query:

```
UPDATE HR.employees
SET salary = salary + (salary * 0.10)
WHERE job_id IN (
    SELECT job_id
    FROM HR.jobs
    WHERE job_id = 'IT_PROG'
);
```

Snapshot of Query:



The screenshot displays a SQL query execution window. The top section, titled 'Query Builder', shows the following SQL query:

```
UPDATE HR.employees
SET salary = salary + (salary * 0.10)
WHERE job_id IN (
    SELECT job_id
    FROM HR.jobs
    WHERE job_id = 'IT_PROG'
);
```

Below the query, the 'Query Result' tab is active, showing the execution output. The output indicates that a table named 'HIGH_SALARY_EMPLOYEES' was created, 15 rows were inserted, 0 rows were updated, and 5 rows were updated.

Task completed in 0.033 seconds

Table HIGH_SALARY_EMPLOYEES created.

15 rows inserted.

0 rows updated.

5 rows updated.

Q14)

SQL Query:

Created and Populated Our Table

```
CREATE TABLE temp_employees AS  
SELECT * FROM HR.employees;
```

DELETING EMPLOYEES FROM OUT TEMPORARY TABLE

```
DELETE FROM temp_employees  
WHERE salary < (  
    SELECT AVG(salary)  
    FROM temp_employees  
);
```

Snapshot of Query:

The screenshot shows a database query tool interface. At the top, there are two tabs: "Worksheet" and "Query Builder". The "Query Builder" tab is active, displaying a SQL query in a text area. The query is: `CREATE TABLE temp_employees AS` followed by `SELECT * FROM HR.employees;` on a new line. Below the query area, there is a toolbar with icons for running the query, saving, and other functions. To the right of the toolbar, it says "Task completed in 0.035 seconds". Below the toolbar, there are two tabs: "Query Result" and "Script Output". The "Query Result" tab is active, displaying the results of the query. The results are as follows:

```
Table HIGH_SALARY_EMPLOYEES created.

15 rows inserted.

0 rows updated.

5 rows updated.

Table TEMP_EMPLOYEES created.
```

Worksheet

Query Builder

```
DELETE FROM temp_employees  
WHERE salary < (  
    SELECT AVG(salary)  
    FROM temp_employees  
);
```



Query Result x



Script Output x



Task completed in 0.035 seconds

Table HIGH_SALARY_EMPLOYEES created.

15 rows inserted.

0 rows updated.

5 rows updated.

Table TEMP_EMPLOYEES created.

55 rows deleted.

Q15)

SQL Query:

```
UPDATE HR.employees
SET department_id = (
    SELECT department_id
    FROM HR.departments
    WHERE department_name = 'MK_MAN'
)
WHERE employee_id = 100;
```


Snapshot of Query:

The screenshot displays a database query builder interface. At the top, there are two tabs: "Worksheet" and "Query Builder". The "Query Builder" tab is active, showing a SQL query in a text area. The query is as follows:

```
SET department_id = (  
  SELECT department_id  
  FROM HR.departments  
  WHERE department_name = 'MK_MAN'  
)  
WHERE employee_id = 100;
```

Below the query area, there is a toolbar with icons for a pin, a pencil, a save icon, a print icon, and a document icon. To the right of these icons, it says "Task completed in 0.05 seconds".

The main area of the interface displays the results of the query execution:

- Table HIGH_SALARY_EMPLOYEES created.
- 15 rows inserted.
- 0 rows updated.
- 5 rows updated.
- Table TEMP_EMPLOYEES created.
- 55 rows deleted.
- 1 row updated.

One-line description

1. What is a subquery in SQL?

A subquery as a mini-query nested inside the main SQL statement/query. It runs first to find a specific piece of information, and then hands that result to the main query.

2. What is the difference between single-row and multiple-row subqueries?

A single-row subquery gives back exactly one specific record. But, a multiple-row subquery returns a whole list of results for your main query to consider/evaluate.

3. What is a correlated subquery?

This is a subquery that relies on data from the main outer query to work. The subquery has to run from scratch for every single row the main query processes as they are tied together.

4. Why are subqueries used in DML statements?

Subqueries are used to update, insert, or delete data when we don't know the exact target values beforehand. They dynamically hunt the right records before applying the changes.

5. What operators are commonly used with multiple-row subqueries?

Operators like IN, ANY, or ALL are used to check how your data compares against the multiple rows returned by your subquery.